Project 2 – Application Risk Classification

1. Project Overview

The aim of this project is to build a machine learning model that can predict whether a software application poses a security risk based on behavioural feature data.

The dataset contains information about application behaviour represented as sparse feature vectors. Each record begins with a risk score followed by a set of feature index–value pairs. These features represent different behavioural characteristics observed from the application.

To convert the problem into a binary classification task, a threshold is applied to the risk score. Applications with a risk score greater than or equal to **0.3** are labelled as **High Risk**, while applications with a score below this threshold are labelled as **Low Risk**.

The overall objective of the project is to:

- Process the raw dataset into a structured format suitable for modelling
- Train a machine learning model capable of predicting application risk
- Evaluate the performance of the model
- Package the trained model so it can be reused for predictions on new data

This project demonstrates the full machine learning workflow from raw data processing through to model deployment.

---

2. Data Processing

The raw dataset is provided as multiple text files containing sparse feature representations.

Each line in the dataset follows the format:

```
risk_score feature_index:value feature_index:value ...
```

The first value represents the risk score assigned to the application. The remaining values represent behavioural features stored in a sparse format.

Since machine learning models require a structured dataset, the raw data must first be transformed into a flat feature matrix. This is done using a feature mapping file that converts feature indices into named feature columns.

The data processing pipeline performs the following steps:

1. Read each raw text file from the dataset directory
2. Parse the risk score and feature index–value pairs from each line
3. Convert sparse feature representations into dense feature vectors
4. Map each feature index to its corresponding feature name
5. Combine all records into a structured dataset suitable for modelling

After processing, the dataset is represented as a feature matrix where each row corresponds to an application and each column corresponds to a behavioural feature.

---

3. Data Cleaning

Basic data cleaning steps were performed to improve data quality and ensure stable model training.

Although the dataset is relatively structured, it is still important to handle potential data issues such as duplicates or extreme values.

The following cleaning steps were applied:

- **Duplicate records were removed** to prevent the model from learning the same observation multiple times.
- **Infinite values were replaced with missing values** to avoid numerical issues.
- **Missing values were filled with zero**, which is reasonable for sparse behavioural features.
- **Extreme outliers were clipped** to a safe numerical range to prevent them from dominating model training.

These steps help make the dataset more robust and reduce the chance of errors during model training.

---

4. Train, Validation and Test Split

To properly evaluate the model, the dataset was split into three separate subsets:

- **Training set – 70%**
- **Validation set – 15%**
- **Test set – 15%**

The training set is used to train the machine learning model.

The validation set is used during development to evaluate the model and monitor its performance while making improvements.

The test set is kept completely separate and is only used for the final evaluation of the model. This helps provide an unbiased estimate of how the model will perform on new, unseen data.

A **stratified sampling approach** was used when creating the splits to ensure that the class distribution (high risk vs low risk applications) remains consistent across all datasets.

---

5. Model Selection

The model chosen for this project is **XGBoost**, which is a gradient boosting algorithm widely used for structured tabular data.

XGBoost was selected because it is known for its strong performance, efficiency, and ability to capture complex relationships between features.

The model works by combining many decision trees to produce a final prediction. Each tree learns to correct the mistakes of the previous trees, which improves overall performance.

The main model parameters used in this project are:

- **Number of trees:** 800
- **Maximum tree depth:** 10
- **Learning rate:** 0.03
- **Subsample ratio:** 0.8
- **Column sample ratio:** 0.8
- **Evaluation metric:** Log Loss

These parameters were chosen to provide a good balance between model complexity and generalisation.

The model outputs a **probability score** representing how likely an application is to be classified as high risk.

---

6. Decision Threshold

Instead of using the default classification threshold of 0.5, a custom threshold was applied to convert probabilities into class labels.

The chosen threshold is **0.45**.

The classification rule is therefore:

- **Probability ≥ 0.45 → High Risk**
- **Probability < 0.45 → Low Risk**

Using a slightly lower threshold helps improve recall while still maintaining strong precision, which is useful in risk detection tasks where missing a risky application may be more costly than incorrectly flagging a safe one.

---

7. Model Performance

The model was evaluated using standard classification metrics including **precision**, **recall**, and **overall accuracy**.

Example performance results on the test dataset were:

- **Precision:** ~0.926
- **Recall:** ~0.979
- **Accuracy:** ~0.91

These results indicate that the model performs well at identifying high-risk applications while maintaining good overall accuracy.

The performance also exceeds the project target of **90% predictive performance**, demonstrating that the model is effective for this classification task.

---

8. Feature Importance

One advantage of using XGBoost is that it provides feature importance scores.

These scores show how much each feature contributes to the model's predictions.

To help interpret the model, a visualisation of the **top 20 most important features** was generated and saved as:

```
feature_importance.png
```

This plot provides insight into which behavioural features are most influential in determining whether an application is considered high risk.

Understanding feature importance can be useful for analysing application behaviour and identifying patterns associated with risky software.

---

9. Model Packaging

After training and evaluating the model, it was packaged so that it can be reused without retraining.

The trained model is saved using Python's **pickle** library as:

```
application_risk_model.pkl
```

Saving the model in this format allows it to be loaded later for inference in a different environment.

A separate inference script was created to demonstrate how the model can be loaded and used to make predictions on new application data.

This script:

1. Loads the saved model file
2. Reads a new input dataset
3. Applies the same feature structure used during training
4. Generates risk predictions using the trained model

This step completes the deployment portion of the machine learning pipeline.

---

10. Conclusion

This project demonstrates the complete development of a machine learning pipeline for application risk classification.

Starting from raw behavioural data, the project includes data processing, cleaning, model development, evaluation, and model deployment.

The final model, based on the XGBoost algorithm, achieves strong predictive performance and is capable of identifying high-risk applications with high precision and recall.

By packaging the trained model and providing a testing script, the solution can easily be reused to evaluate new applications in the future.

Overall, the project successfully meets the objectives of building and evaluating a machine learning model for application risk prediction.